

and pay according to their usage. Moreover, users can purchase additional capabilities to expand the plug-and-play functions of their existing agents. This allows users to gradually upgrade their agents. Within the MetaGPT framework, AgentStore can support the collaboration of various agents. Users can collect several agents together to carry out more complex tasks or projects, and all the agents share and comply with development and communication protocols defined in MetaGPT.

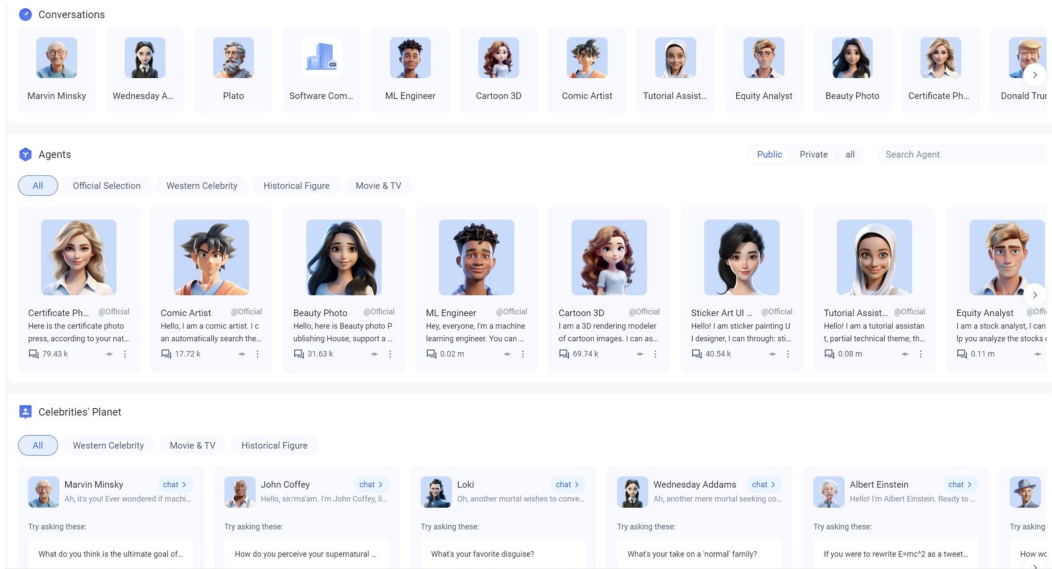


Figure 6: AgentStore is a platform dedicated to serving users in the creation and development of agents within the MetaGPT framework. This platform provides users with an operational interface, allowing users to easily manage a variety of agents with different emotions, personalities, and capabilities for specific tasks.

B A DEMO OF THE EXECUTION

In this section, we outline the complete process of software development using MetaGPT. It begins with a user's input command (as shown in Appendix B.1) and ends with software designed according to the user's specifications.

B.1 USER INPUT

Upon receiving an instruction from the user, MetaGPT collaborates with a professional development team to fulfill the task. Here is a demo of user input:

User input

```
I write a python3 GUI app such that you can draw an image with it
```

B.2 METAGPT DEVELOPMENT PROCESS

Now we provide a step-by-step explanation of the standardized output process for each agent.

Product Manager The Product Manager generates a Product Requirement Document (PRD), as detailed in the specified documentation. This document encompasses goals, user stories, competitive analysis, requirement analysis and requirement pool. Additionally, a competitive quadrant chart is produced (see Figure 7). Subsequently, these documents and charts are handed over to the architect for system design.

Product Requirement Document

```
1 ## Original Requirements
2 The boss requires a Python3 GUI color meter that outputs the RGB values when it moves.
3
4 ## Product Goals
5 ```python
6 [
7     "Create a user-friendly GUI color meter",
8     "Ensure the color meter accurately outputs RGB values",
9     "Ensure the color meter updates RGB values in real-time as it moves"
10 ]
11 ```
12
13 ## User Stories
14 ```python
15 [
16     "As a user, I want to select any color on the screen, so that I can get its RGB values",
17     "As a user, I want the RGB values to update in real-time as I move the color meter, so that I
18     ↳ can get accurate color information",
19     "As a user, I want a simple and intuitive interface, so that I can easily use the color meter"
20 ]
21 ```
22
23 ## Competitive Analysis
24 ```python
25 [
26     "Color Cop: A popular color picker for Windows. It's simple and easy to use, but lacks real-
27     ↳ time RGB value updates",
28     "Just Color Picker: Offers real-time color picking and multiple color code formats. However,
29     ↳ its interface is cluttered and not very user-friendly",
30     "Instant Eyedropper: A simple color picker that lives in the system tray. It's convenient but
31     ↳ lacks advanced features",
32     "ColorZilla: A color picker extension for browsers. It's convenient for web design but not
33     ↳ suitable for desktop applications",
34     "ColorPic: Offers color palettes and mixer tools. It's feature-rich but can be overwhelming
35     ↳ for simple tasks"
36 ]
37 ```
```

Product Requirement Document

```

1
2 ## Requirement Analysis
3 The product should be a Python3 GUI application that serves as a color meter. It should allow
  ↳ users to select any color on the screen and display the corresponding RGB values. The RGB
  ↳ values should update in real-time as the color meter moves. The interface should be
  ↳ simple and user-friendly.
4
5 ## Requirement Pool
6 ```python
7 [
8     ("Design a user-friendly GUI for the color meter", "P0"),
9     ("Implement color selection functionality", "P0"),
10    ("Display RGB values of the selected color", "P0"),
11    ("Update RGB values in real-time as the color meter moves", "P0"),
12    ("Test the application for accuracy and performance", "P1")
13 ]
14 ```
15
16 ## UI Design draft
17 The UI should be minimalistic and intuitive. It should primarily consist of a color meter, which
  ↳ could be a magnified area of the screen that the user can move around. The RGB values of
  ↳ the currently selected color should be prominently displayed, updating in real-time as
  ↳ the color meter moves. The layout should be clean and uncluttered, with a focus on
  ↳ functionality and ease of use.
18
19 ## Anything UNCLEAR
20 There are no unclear points.

```

Reach and engagement of campaigns

Figure 7: The quadrant chart for the “Drawing App” generated by the *product manager* agent.

Architect Based on the requirements in PRD, the Architect agent devises technical specifications including system architecture diagrams and interface definitions. Initially, the Architect defines the overarching technical trajectory. Subsequently, the project’s architecture, including files, classes (Figure 8) and the sequence flow chart (Figure 12), is designed. The Architect’s documentation is then given to the project manager for task allocation and execution.

System design

```

1  ## Implementation approach
2  We will use Python's Tkinter library to create the GUI for the color meter. Tkinter is a standard
   ↳ Python interface to the Tk GUI toolkit and is included with most Python installations. It
   ↳ is simple to use and powerful enough for our needs.
3
4  The color selection functionality will be implemented using the PIL (Pillow) library, which allows
   ↳ us to capture the screen's pixel data. By moving the mouse, we can get the RGB values of
   ↳ the pixel under the mouse cursor in real-time.
5
6  The RGB values will be displayed on the GUI using Tkinter's Label widget. We will use Python's
   ↳ threading library to update the RGB values in real-time as the color meter moves.
7
8  The application will be tested for accuracy and performance using Python's unittest library.
9
10 ## Python package name
11 ``python
12 "color_meter"
13 ```
14
15 ## File list
16 ``python
17 [
18     "main.py",
19     "color_picker.py",
20     "gui.py",
21     "tests.py"
22 ]
23 ```
24
25 ## Anything UNCLEAR
26 The requirement is clear to me.

```

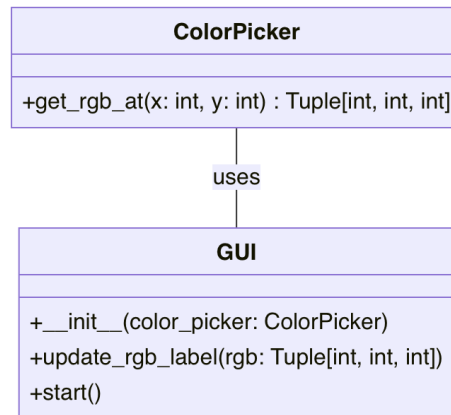
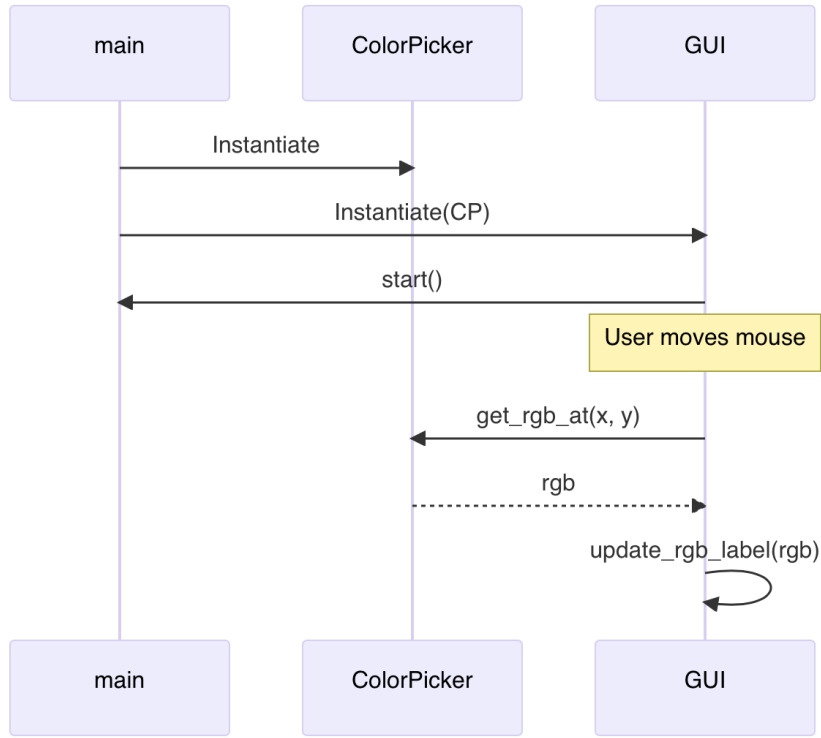


Figure 8: Data structures and interface definitions for the “Drawing App” generated by the *architect* agent.

Project Manager The Project Manager breaks down the project into a task list. Furthermore, each code file is analyzed based on its intended functionality and then treated as a separate task assigned to Engineers.

Figure 9: The program call flow for the “Drawing App” generated by the *architect* agent.

Tasks

```

1  ## Required Python third-party packages
2
3  ```python
4  """
5  tkinter==8.6
6  Pillow==8.3.1
7  """
8  ```
9
10 ## Required Other language third-party packages
11 ```python
12 """
13 No third-party packages required in other languages.
14 """
15 ```
16
17 ## Full API spec
18 ```python
19 """
20 No APIs are used in this project.
21 """
22 ```
23
24 ## Logic Analysis
25 ```python
26 [
27     ("main.py", "Contains the main application loop and initializes all the other components."),
28     ("canvas.py", "Implements the drawing canvas, responds to mouse events, and uses the selected
29         ↳ tool and color."),
29     ("tools.py", "Implements the tool selection interface."),
30     ("color_picker.py", "Implements the color selection interface."),
31     ("file_manager.py", "Implements the file saving and opening functionality.")
32 ]
33 ```

```